

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A Project Proposal
on
“Biruwa”

[Code No: COMP 207]

(For partial fulfillment of II/II Year/Semester in Computer Science/Engineering)

Submitted by:

Suraj Bhattarai (6)
Ashitom Budhathoki (9)
Anmol Dahal (11)

Submitted to:

Mr. Suman Shrestha
Department of Computer Science and Engineering

Submission Date: 18th September 2024

Abstract

Early detection of plant diseases is essential not only for professionals in agriculture but also for everyday individuals who engage in gardening and plant care. Many people struggle with identifying plant diseases promptly, which can lead to further damage and loss of plants. This project focuses on developing an easy-to-use plant health analysis application that leverages image recognition with (Supervised learning, n.d.) and deep learning techniques to help anyone quickly and accurately diagnose plant diseases. The application will offer a reliable solution for users to monitor the health of their plants and take timely actions to prevent further damage. The machine learning model is implemented with help of existing models and employs supervised learning to analyze images and detect diseases based on plant symptoms. It will be trained on a diverse set of labeled plant disease data to ensure precise identification. The front-end of the application will be built using Flutter, offering a simple and intuitive mobile interface. Users can easily capture images of their plants and receive immediate feedback on plant health. The expected outcome is a user-friendly mobile application capable of identifying various plant diseases providing real-time insights and recommendations for care. In conclusion, this project provides a practical and accessible solution for plant disease detection. Future improvements may include expanding the range of plants and diseases covered, improving accuracy and adding features to enhance plant care knowledge.

Keywords: *Plant health, Deep learning, Supervised learning, Flutter*

Table of Contents

Abstract	i
List of Figures	iv
Acronyms	v
Chapter 1 Introduction	1
1.1 Background	1
1.2 Objectives.....	2
1.3 Motivation and Significance	2
Chapter 2 Related Works	3
2.1 Plantix.....	3
2.1.1 Key Features	3
2.2 Agroi.....	4
2.2.1 Key Features	4
Chapter 3 Procedure and Methodology	5
3.1 Requirement Analysis and Planning	5
3.2 Data Collection and Preparation	6
3.3 Model Development and Training	7
3.4 Application Development	8
3.5 Testing, Maintenance, and Validation.....	9
3.6 Documentation and Training.....	9
Chapter 4 System Requirement Specifications	10
4.1 Software Specifications.....	10
4.1.1 Functional Requirements	10
4.1.2 Non-Functional Requirements	10
4.1.3 Technical Stack and Resources.....	11

4.2 Hardware Specifications	12
4.2.1 Development Hardware	12
4.2.2 Deployment Hardware	12
Chapter 5 Project Planning and Scheduling.....	13
5.1 Tasks.....	13
References	14

List of Figures

Figure 2.1.1 Plantix.....	3
Figure 2.2.1 Agroi.....	4
Figure 3.1.1 Planning Process.....	5
Figure 3.1.2 Use Case Diagram.....	6
Figure 3.2.1 Sample Data Showing Healthy and Unhealthy Plant.....	6
Figure 3.3.1 CNN Architecture.....	7
Figure 3.3.2 ResNet Architecture.....	7
Figure 3.4.1 Mock Application Design.....	8
Figure 5.1 Gantt Chart.....	13

Acronyms

The list of all acronyms used in the documentation is included in this section.

AI: Artificial Intelligence

CNN: Convolutional Neural Network

UI: User Interface

ResNet: Residual Network

iOS: iPhone Operating System

REST: Representational State Transfer

API: Application Programming Interface

GPU: Graphical Processing Unit

RAM: Random Access Memory

GB: Gigabyte

Chapter 1 Introduction

1.1 Background

In the rapidly advancing field of plant health management, the demand for efficient, accurate, and easily accessible diagnostic tools has never been greater, driven by the need to quickly address plant diseases and ensure optimal plant health in diverse and changing environments. Traditional methods of plant disease identification often rely on manual inspection and expert knowledge, which can be time-consuming and prone to error, especially for non-experts. Recognizing these challenges, the development of Biruwa aims to transform plant health management by integrating advanced technology to offer a streamlined, user-friendly solution for diagnosing plant diseases.

The evolution of plant health technology has progressed from basic visual inspections to sophisticated digital solutions involving machine learning and image recognition. However, many existing tools remain fragmented, focusing on specific aspects of disease detection without offering a comprehensive, accessible platform. As the need for effective plant management solutions grows, there is a clear opportunity to unify these technologies into a single, intuitive application like Biruwa. By addressing the limitations of previous innovations and incorporating real-time diagnostic capabilities, Biruwa seeks to set a new standard in plant health monitoring.

Biruwa leverages advanced features such as image recognition and machine learning to provide accurate and instant feedback on plant health. Designed for ease of use, Biruwa allows users to simply capture images of their plants and receive reliable diagnostic results and recommendations. As the field of plant health management continues to evolve, Biruwa positions itself as a tool that not only meets current needs but also anticipates future advancements. By integrating cutting-edge technology into a user-friendly platform, Biruwa aims to enhance plant care and contribute to more effective and accessible plant health management solutions.

1.2 Objectives

The primary objective of this project is to enhance plant health management through advanced technology. We initiated this project with the following objectives:

- Develop a user-friendly mobile application for capturing and diagnosing plant diseases through image recognition.
- Provide actionable recommendations for managing identified plant diseases.
- Continuously expand and improve the application with user feedback and updated plant disease data.

1.3 Motivation and Significance

The motivation behind Biruwa stems from the widespread challenges faced by individuals in maintaining plant health, particularly in identifying and managing plant diseases. Traditional methods of plant disease diagnosis often rely on manual inspection and expert knowledge, which can be time-consuming and difficult for casual gardeners or those without specialized training. This gap in accessible, real-time diagnostic tools can lead to delayed interventions and increased plant damage, affecting not only personal gardens but also broader agricultural practices.

Biruwa seeks to address these issues by providing an innovative solution that leverages advanced image recognition and machine learning technologies. The application enables users to quickly capture images of their plants and receive instant feedback on potential diseases, complete with actionable recommendations. By enhancing the ability to promptly identify and address plant health issues, Biruwa not only improves individual plant care but also supports broader goals of sustainable agriculture and food security. This dual focus on practical application and technological advancement underscores Biruwa's commitment to transforming plant health management and fostering a more informed and proactive approach to plant care.

Chapter 2 Related Works

In the realm of plant health diagnostics and mobile applications, there is considerable potential for innovation and improvement. In this context, our exploration identified several existing projects that align closely with our objectives:

2.1 Plantix

(Plantix, n.d.) is a widely used app for plant disease diagnosis. Farmers can upload images of infected plants, and the app uses machine learning to detect the disease and suggest remedies.

2.1.1 Key Features

- Image-based plant disease detection using AI.
- Crop advisory services based on weather conditions.
- Community-based solutions where farmers can share information.



Figure 2.1.1 Plantix

2.2 Agroi

(Agroi, n.d.) is a cutting-edge agricultural technology platform designed to optimize farming practices and enhance crop management. By leveraging data analytics, machine learning, and real-time monitoring, Agroi provides farmers with actionable insights to improve productivity and sustainability.

2.2.1 Key Features

- Combines AI models with drones to capture images across vast areas.
- Suggests treatment based on crop disease detection.
- Offers an integrated platform with weather and soil data analysis.

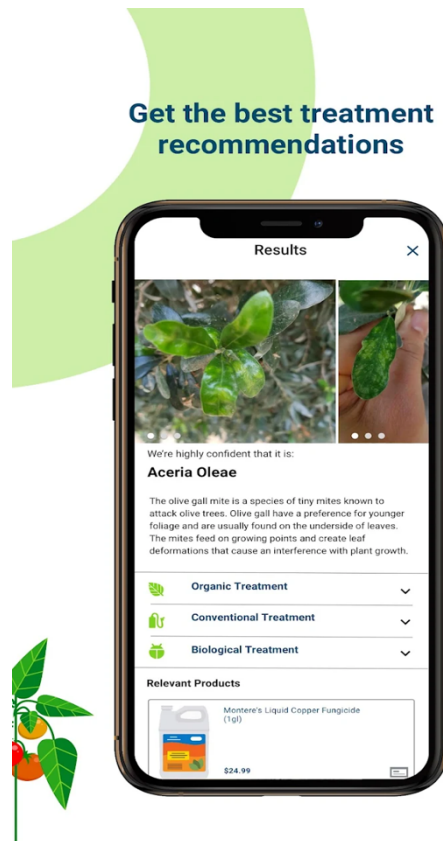


Figure 2.2.1 Agroi

Chapter 3 Procedure and Methodology

In this chapter, we outline the sequential process guiding the development of Biruwa. We'll discuss how we approach different aspects of the project, ensuring a systematic and effective application.

3.1 Requirement Analysis and Planning

The initial phase involves defining the primary objectives and functionalities of Biruwa. This includes specifying features such as image capture, disease detection, and treatment recommendations. To ensure that the application meets user needs, we will conduct surveys and interviews with potential users, including gardeners and agriculturalists, to gather insights into their specific requirements and preferences. This phase also includes determining and documenting detailed use cases that outline how different users will interact with the application, such as capturing plant images and receiving diagnoses. Additionally, a review of existing plant health diagnostic tools will help identify gaps and opportunities for improvement, guiding the development process.



Figure 3.1.1 Planning Process

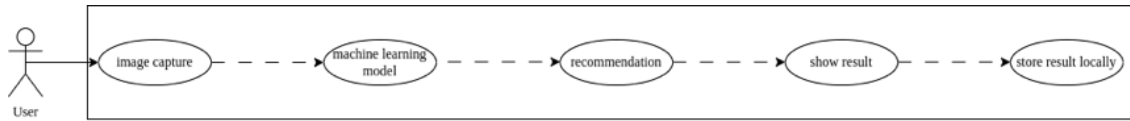


Figure 3.1.2 Use Case Diagram

3.2 Data Collection and Preparation

The next step is to gather a comprehensive dataset of plant images and disease labels. This dataset can be sourced from agricultural research institutions, online repositories, or through collaborations with plant health experts. Once collected, the data will undergo preprocessing to enhance its quality and usability. This includes annotating images with disease labels, normalizing image sizes, and applying data augmentation techniques to increase the dataset's diversity and robustness. Proper data preparation is crucial for training effective machine learning models. Additionally, continuous updating of the dataset with new images and disease information will help maintain model accuracy and relevance.



(a)



(b)

Figure 3.2.1: Sample data showing (a) Unhealthy plant (b) Healthy plant

3.3 Model Development and Training

With the data prepared, the next step is to select and develop suitable machine learning algorithms for image recognition tasks. While (Convolutional neural network, n.d.) are a common choice for visual data processing, the specific algorithm to be used will depend on the findings of our research. Potential candidates include CNNs, ResNet, or other advanced architectures that may offer better performance based on our evaluation. The development process will involve training these models using the prepared dataset, which will be split into training, validation, and test sets to ensure comprehensive evaluation. Hyperparameters will be tuned to optimize model performance, and techniques such as cross-validation will be employed to improve accuracy and prevent overfitting. Ongoing evaluation and refinement will be crucial to achieving high efficiency in disease detection and adapting to new data and insights.

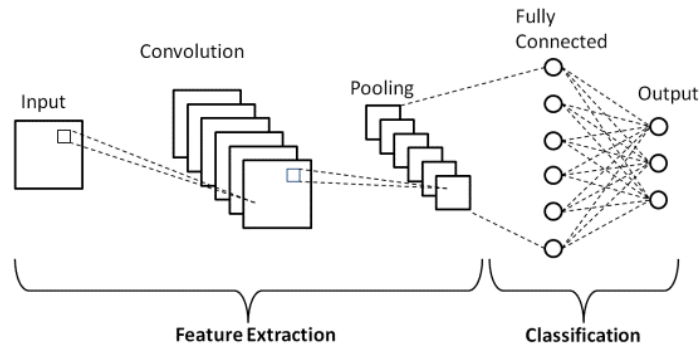


Figure 3.3.1 CNN Architecture

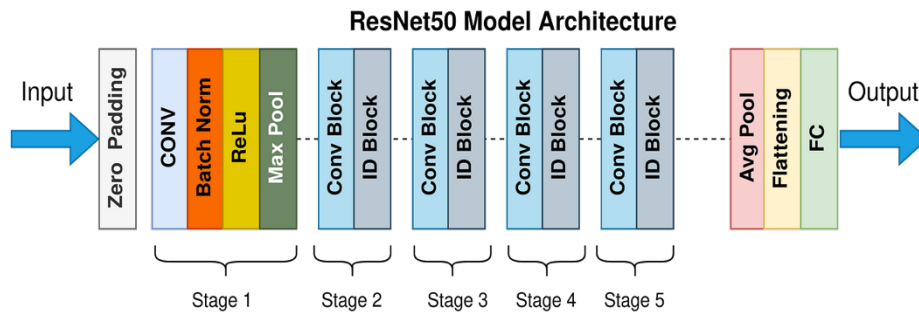


Figure 3.3.2 RestNet Architecture

3.4 Application Development

In parallel with model development, the mobile application will be designed and developed using Flutter to ensure a seamless, cross-platform experience. The UI will be meticulously crafted to be intuitive and user-friendly, incorporating screens for capturing plant images, displaying disease detection results, and offering treatment recommendations. The application will integrate the trained machine learning models to enable real-time image analysis and provide immediate feedback to users. Essential features will include image capture via the device camera, automatic upload, and processing through the machine learning model, with diagnostic results and treatment suggestions displayed clearly. Additionally, the app will support functionalities such as saving and reviewing historical data. This iterative development approach will ensure that the application is responsive, effective, and aligned with user needs, ultimately enhancing plant health management for a wide range of users.

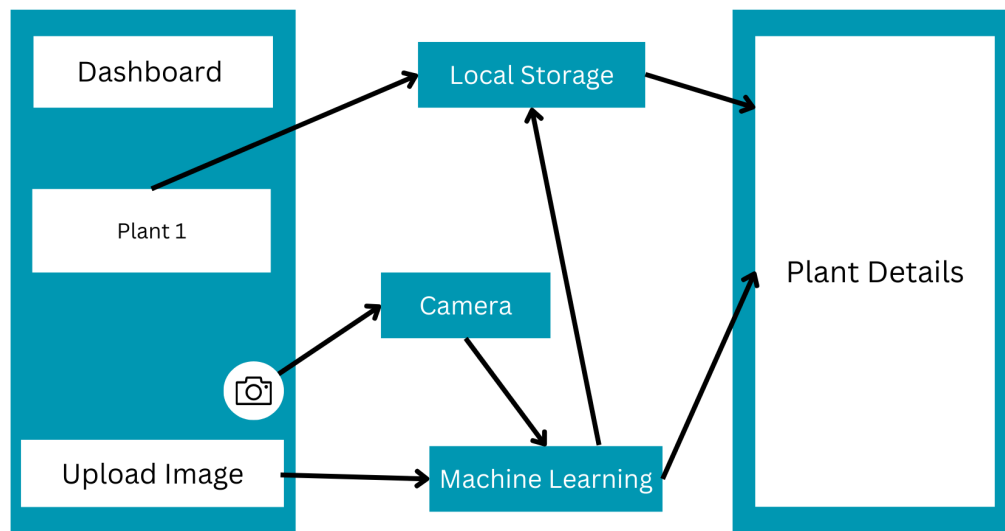


Figure 3.4.1: Mock Application Design

3.5 Testing, Maintenance, and Validation

Once the application is developed, rigorous testing will be conducted to ensure its functionality and reliability. Functional testing will verify that all features, such as image capture, disease detection, and treatment recommendations, operate as intended. Usability testing will involve beta users to assess the application's ease of use and effectiveness in diagnosing plant diseases. Feedback from these tests will be used to make necessary improvements, refine user interfaces, and enhance overall performance. Following deployment, continuous maintenance will involve monitoring the application's performance, updating it with new features, fixing bugs, and incorporating user feedback. Regular updates will also address emerging plant diseases and incorporate the latest research findings. Additionally, the machine learning model will be regularly reviewed and improved based on new data and user interactions to enhance its efficiency and accuracy. This ongoing support is crucial for maintaining high performance and user satisfaction over time, ensuring that the application remains a valuable tool.

3.6 Documentation and Training

Comprehensive documentation will be created for both users and developers to ensure effective use and maintenance of the application. User guides will include detailed instructions on utilizing key features like image capture, disease detection, and result interpretation, along with troubleshooting tips. For developers, technical specifications and system architecture documentation will be provided to facilitate future updates and maintenance. Training resources, including video tutorials and interactive sessions, will be developed to help users understand and leverage the application effectively for plant health management. Continuous feedback from users will be used to refine these materials, ensuring that they are both practical and user-friendly, thereby supporting the smooth adoption and optimal utilization of our application.

Chapter 4 System Requirement Specifications

This chapter outlines the requirements for the proposed system, detailing both the software and hardware specifications necessary for development and deployment.

4.1 Software Specifications

This section describes the software system to be developed, encompassing both functional and non-functional requirements, as well as the tools and libraries to be used.

4.1.1 Functional Requirements

The following functional requirements outline the core capabilities and features that the application must provide to fulfill its purpose effectively. These requirements ensure that the system meets the essential needs of its users and supports all intended functionalities.

- **Image Capture:** Ability to capture images of plants
- **Health Prediction:** Integration of machine learning models for health prediction
- **Recommendation:** Provision of actionable treatment suggestion based on plant condition.
- **User Interface:** Intuitive and user-friendly interface for easy navigation and interaction with application.

4.1.2 Non-Functional Requirements

The following non-functional requirements describe the overall qualities and performance criteria of the application, ensuring that it operates smoothly.

- **Performance:** Efficient performance and analysis
- **Scalability:** Capability to handle an increasing number of users and data without performance degradation.
- **Cross-Platform Compatibility:** Support for iOS and Android operating systems

4.1.3 Technical Stack and Resources

The successful development and deployment of the application will rely on a carefully selected set of tools and libraries, each chosen for their specific roles in addressing various aspects of the project. This section details the key technologies and resources to be utilized, spanning from front-end design and back-end integration to machine learning implementation. By employing these tools, we aim to ensure a robust, efficient, and effective application that meets project objectives and delivers a high-quality user experience.

4.1.3.1 Front End Tools

We will be using following front-end tools for development of our application.

- Flutter: (Flutter: Build apps for any screen, n.d.) is a framework for developing the mobile application with a unified codebase for both iOS and Android.
- Dart: (Dart Language Overview, n.d.) is a programming language that is used with Flutter to build the user interface and application logic.
- Figma: (Figma: Design systems and collaboration, n.d.) is a tool for designing user interface prototype.

4.1.3.2 Back End Tools

We will be following back-end tools for development of our application.

- REST API (Belouche, n.d.): Protocol for communicating between mobile application and the machine learning model, facilitating data exchange and results retrieval.

4.1.3.3 Machine Learning Development

We will be using following tools for implementing machine learning.

- Python: Programming language primarily used for machine learning and data analysis.
- Tensorflow/Keras: (TensorFlow: Open source platform for machine learning, n.d.) and (Keras , n.d.) are the libraries for building and training machine learning model.

4.2 Hardware Specifications

To support both the development and deployment phases of the project, it's essential to specify the hardware requirements that will ensure optimal performance. This section outlines the necessary hardware components for development

4.2.1 Development Hardware

- Processor: Multi-core processor (Intel/AMD/M1) for smooth development.
- Ram: Minimum of 8 GB for handling development tasks.
- Storage: At least 256 GB to store project files, development tools and large datasets.
- Graphics Card: Dedicated GPU for accelerated model training and image processing.

4.2.2 Deployment Hardware

Since flutter is a relatively newly built toolkit, Android API 19 and above or iPhone Operating System (iOS) 8 and above will be required to support the application's features. To run the application smoothly, at least 2 Gigabyte (GB) Random Access Memory (RAM) and 1 GB of free storage would most likely be needed.

Chapter 5 Project Planning and Scheduling

The project's timeline is depicted in the Gantt chart below, showcasing the planned start and end dates for each phase, major milestones, and the overall project duration.

Tasks/ Week	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Requirement Analysis & Planning												
Data Collection & Preperation												
Model Development & Training												
Application Development												
Testing, Validation & Maintenance												
Documentation & Training												

Figure 5.1 Gantt Chart

5.1 Tasks

1. Requirement Analysis & Planning (Week 1): Conduct thorough analysis and create a detailed project plan with scope, timeline, and risk management.
2. Data Collection & Preparation (Week 2): Gather dataset through various sources.
3. Model Development & Training (Week 3-8): Develop and Train Machine Learning Model.
4. Application Development (Week 7-9): Create Front End Application and API To connect application with Machine learning model.
5. Testing, Validation & Maintenance (Week 10-11): Test and validate features and main the application
6. Documentation (Week 12): Create documentation for application.

References

- Belouche, Y. (n.d.). *Building a REST API for a Machine Learning Model and Mobile App Integration*. Retrieved from Medium: <https://medium.com/codex/building-a-rest-api-for-a-machine-learning-model-and-mobile-app-integration-part1-417a0d8c3df8>
- Convolutional neural network*. (n.d.). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Convolutional_neural_network
- Dart Language Overview*. (n.d.). Retrieved from Dart: <https://dart.dev>
- Figma: Design systems and collaboration*. (n.d.). Retrieved from Figma: <https://www.figma.com>
- Flutter: Build apps for any screen*. (n.d.). Retrieved from Flutter: <https://flutter.dev>
- Keras Documentation*. (n.d.). Retrieved from Keras: <https://keras.io>
- New Plant Diseases Dataset*. (2018). Retrieved from Kaggle: <https://www.kaggle.com/datasets/vipooooool/new-plant-diseases-dataset>
- Plantix*. (n.d.). Retrieved from Plantix: <https://plantix.net/en/>
- Supervised learning*. (n.d.). Retrieved from Wikipedia : https://en.wikipedia.org/wiki/Supervised_learning
- TensorFlow: Open source platform for machine learning*. (n.d.). Retrieved from Tensorflow: <https://www.tensorflow.org>